

Training Loops, Debugging, and Visualization

Department of Statistics

Oregon State University

05/01/2026

What we'll cover

01

PyTorch DataLoaders

Dataset vs DataLoader, batching, shuffling, workers

02

Debugging Neural Networks

Exploding/vanishing gradients, activation inspection, gradient flow

03

Visualization Toolkit

Activations, gradients, weights, and loss curves

04

Weights & Biases

Logging, and checkpoints

05

FashionMNIST: MLP vs CNN

Comparing experiment runs using W&B

PyTorch DataLoaders

Dataset vs DataLoader



Dataset

What does a single sample look like?

- Defines `__len__` and `__getitem__`
- Owns the on-disk or in-memory data
- Applies per-sample transforms (resize, normalize)
- Knows nothing about batching or order of samples*



DataLoader

How do samples flow into the model?

- Wraps a Dataset and yields batches
- Handles shuffling, batching, and parallelization
- Uses workers for async I/O
- `collate_fn` for variable-length data
- Pins memory for fast CPU->GPU transfer (Allocates batches in page-locked RAM)

*: can define order of samples in the dataset, if not shuffling in DataLoader

Dataset Example:

```
class ImageFolderDataset(Dataset):
    def __init__(self, root_dir):
        # "Owns" the data by holding paths to it
        self.image_paths = sorted(glob.glob(f"{root_dir}/*.jpg"))
        self.labels = [...] # parsed from filenames or a CSV

    def __getitem__(self, idx):
        # Reads from disk on demand
        image = Image.open(self.image_paths[idx])
        return image, self.labels[idx]
```

On-disk data

```
class TensorDataset(Dataset):
    def __init__(self, features, labels):
        # "Owns" the data by holding the tensors directly
        self.features = features # shape: (N, D)
        self.labels = labels # shape: (N,)

    def __getitem__(self, idx):
        return self.features[idx], self.labels[idx]
```

In-memory data

Dataset Example:

```
class ImageDataset(Dataset):
    def __init__(self, image_paths, labels, transform=None):
        self.image_paths = image_paths
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        # Load the image (lazy loading - only when requested)
        image = Image.open(self.image_paths[idx]).convert('RGB')
        label = self.labels[idx]

        # Apply transforms (augmentation, normalization, ToTensor)
        if self.transform:
            image = self.transform(image)

        return image, label
```

DataLoader: the essentials

fashion_mnist.py

```
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

tfm = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.286,), (0.353,)) # standardize data
])

train_ds = datasets.FashionMNIST(
    root='data', train=True, download=True,
    transform=tfm)

train_loader = DataLoader(
    train_ds,
    batch_size=64,
    shuffle=True,
    num_workers=4,
    pin_memory=True)
```

batch_size

Samples per forward/backward. Bigger = smoother gradients, more GPU memory.

shuffle

Reorders each epoch. Critical for SGD; turn OFF for validation.

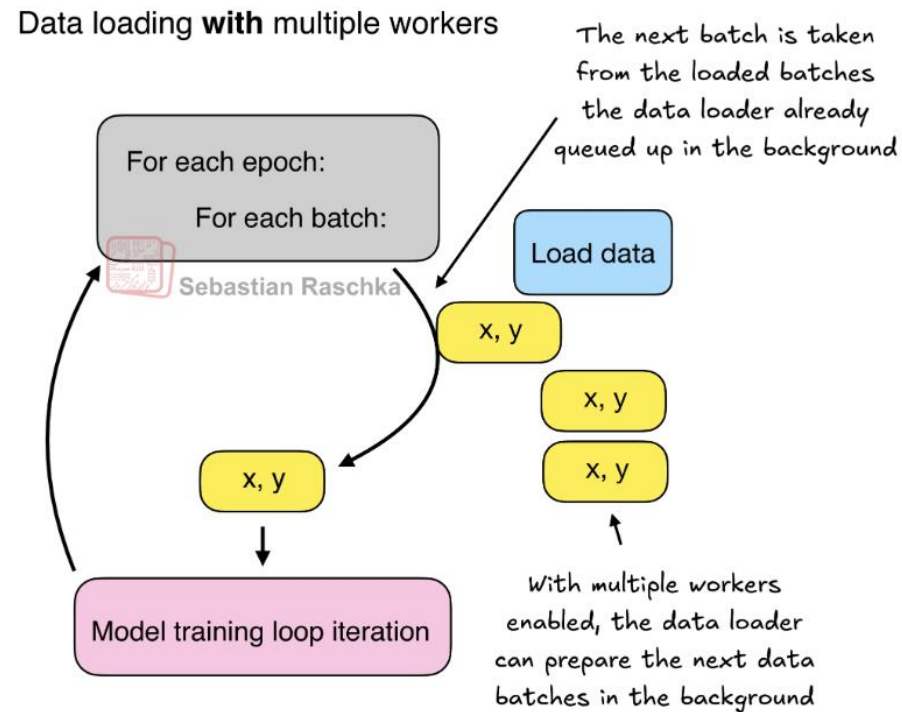
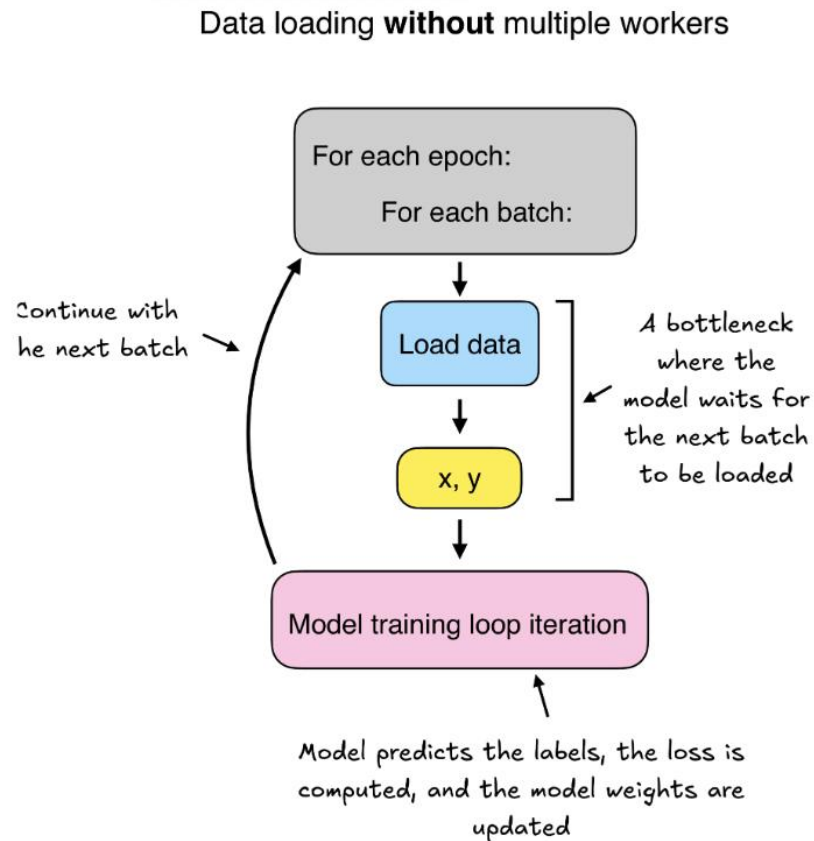
num_workers

Parallel data-loading processes. Rule of thumb: 2–8 on a typical box.

pin_memory

Stages tensors in pinned host RAM for faster GPU transfer.

Loading data without multiple workers



- When working with small datasets, increasing `num_workers` may not provide speed up because of overhead
- For Jupyter notebooks, setting `num_workers > 0` can sometimes lead to issues related to the sharing of resources between different processes.

Debugging neural networks

When training silently goes wrong



Loss is NaN

Gradients exploded (weights too large); one update overflowed and propagating NaN through subsequent forward passes.



Loss won't change

Vanishing gradients (especially for early layers), ReLUs activation remains silence, or learning rate too small.



Train $\uparrow$, Validation $\downarrow$

Overfitting, model memorizes training data, doesn't generalize; Might need regularization or data augmentation



Loss looks OK, accuracy not good

Silent bugs, wrong loss function, label mismatch, class imbalance

Five tools to stabilize gradient flow

A. PREVENTIVE

1

Parameter initialization *Xavier(tanh) / He (ReLU)* Demo

pick $\text{Var}(W)$ so variance of *activations* is preserved through layers
 $W \sim N(0, 1/n_{\text{in}})$ / $W \sim N(0, 2/n_{\text{in}})$

2

Non-saturating activations *ReLU, GELU, SiLU*

Derivatives not vanishing; no sigmoid squash. GELU/SiLU keep small negative gradients to avoid dead units.

3

Normalization layers *LayerNorm, BatchNorm*

Re-center and re-scale activations each layer. LayerNorm (per-sample across features) or BatchNorm (per-feature across batch).

4

Residual connections $y = x + F(x)$

Identity shortcut gives gradients a bypass. Foundation of ResNet, Transformer etc.

B. CORRECTIVE

5

Gradient clipping *global-norm / value*

Rescales ∇L whenever $\|\nabla L\| > \tau$. Prevents a single bad batch from destroying the parameter state. Essential in language-model pretraining where rare tokens cause gradient spikes.

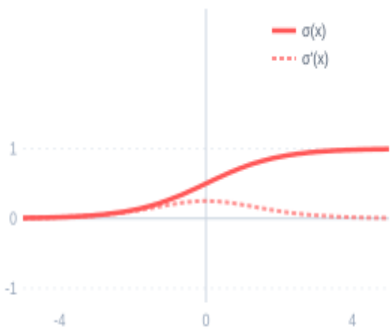
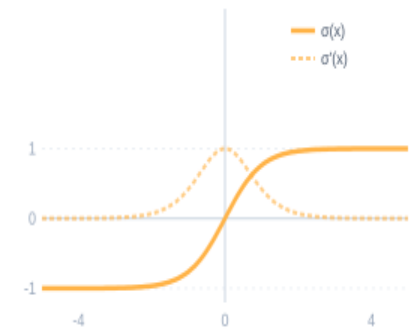
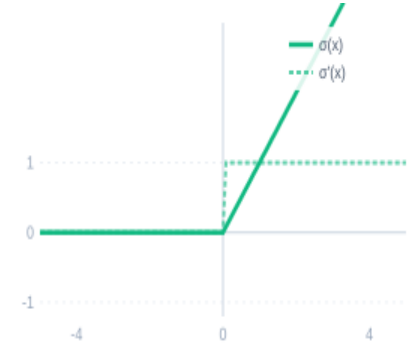
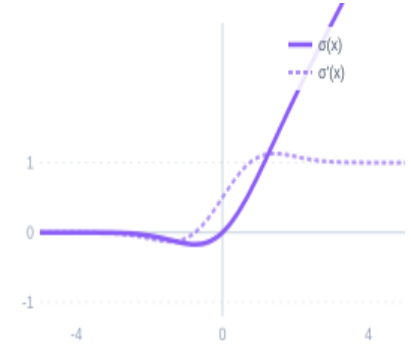
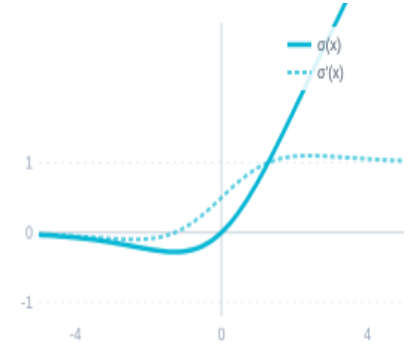
$$\text{if } \|\nabla L\|_2 > \tau: \quad \nabla L \leftarrow \nabla L \cdot \tau / \|\nabla L\|_2$$

Choose τ so that $\|\nabla L\|$ is clipped in roughly 5–10% of steps

always init + use ReLU/GELU + normalize. Add residuals once depth > 10. Enable clipping when you see gradient spikes.

Choosing $\sigma(\cdot)$

Each activation has a derivative profile that determines whether the gradient survives the backward pass.

Sigmoid	Tanh	ReLU	GELU	SiLU
$\sigma(x) = 1/(1+e^{-x})$  Properties Range (0,1) · max $\sigma' = 0.25$ Vanishes in deep nets	$\tanh(x)$  Properties Range (-1,1) · zero-centered · max $\sigma' = 1$ Better than sigmoid, still saturates	$\max(0, x)$  Properties $\sigma' \in \{0, 1\}$ · cheap · sparse Default. Watch for dead units.	$x \cdot \Phi(x)$  Properties Smooth · small negative gradient Default in transformers (BERT, GPT-2)	$x \cdot \sigma(x)$  Properties Smooth · self-gated · cheaper than GELU Default in modern LLMs (Llama, Mistral)

Rule of thumb: ReLU for CNNs and most MLPs · GELU/SiLU for transformers · avoid sigmoid/tanh in hidden layers of deep NN. sigmoid at the output layer (binary classification, gating).

BatchNorm vs LayerNorm

Both layers compute $\widehat{x}_{ij} = (x_{ij} - \mu) / \sqrt{\sigma^2 + \varepsilon}$, then learn an affine rescale $\gamma \widehat{x}_{ij} + \beta$. They differ only in the index set used to estimate the moments.

BatchNorm — average across samples

$$\begin{aligned}\hat{\mu}_j &= \frac{1}{B} \sum_{i=1}^B x_{ij} \\ \hat{\sigma}_j^2 &= \frac{1}{B} \sum_{i=1}^B (x_{ij} - \hat{\mu}_j)^2\end{aligned}$$

Per feature j , estimated from the B samples in the mini-batch.

ESTIMATOR

Sample mean / variance of feature j across the batch

I.I.D. ASSUMPTION

Treats samples in a batch as i.i.d. draws. Tiny batches give noisy moments and unstable training.

TRAIN VS TEST

At test time we don't have a batch. Use running averages of mean and variance accumulated during training. (needs to specify mode of the model)

LayerNorm — average across features

$$\begin{aligned}\hat{\mu}_i &= \frac{1}{d} \sum_{j=1}^d x_{ij} \\ \hat{\sigma}_i^2 &= \frac{1}{d} \sum_{j=1}^d (x_{ij} - \hat{\mu}_i)^2\end{aligned}$$

Per sample i , estimated from the d features within that sample.

ESTIMATOR

A single sample's empirical moments across its d coordinates

NO I.I.D. REQUIREMENT

Independent of batch size; the statistic is computed within the sample itself.

TRAIN VS TEST

Same operation in both phases. No running averages, no train/test discrepancy. This is why it dominates in transformers and online inference.

Peek inside with forward hooks

Hooks let you snapshot any module's input or output during a forward pass — without changing model code.

- ✓ Pre-Activations roughly centered, std ≈ 1
- ⚠ Saturated tanh / sigmoid → vanishing gradient
- ⚠ All zeros after ReLU → dead neurons
- ✓ Variance stable across layers

```
hooks.py

activations = {}

def save_act(name):
    def hook(module, input, output):
        activations[name] = output.detach()
    return hook

# attach to every Linear / Conv2d
for name, m in model.named_modules():
    if isinstance(m, (nn.Linear, nn.Conv2d)):
        m.register_forward_hook(save_act(name))

_ = model(x_batch) # populate activations

for name, a in activations.items():
    print(name, a.mean().item(), a.std().item())
```

Four things worth watching



Loss & metrics curves

Train vs validation loss over epochs. The single most informative plot.



Activation distributions

Histograms per layer. Watch for saturation or zero collapse.



Gradient histograms

Gradient norm per layer. Catches exploding/vanishing early.

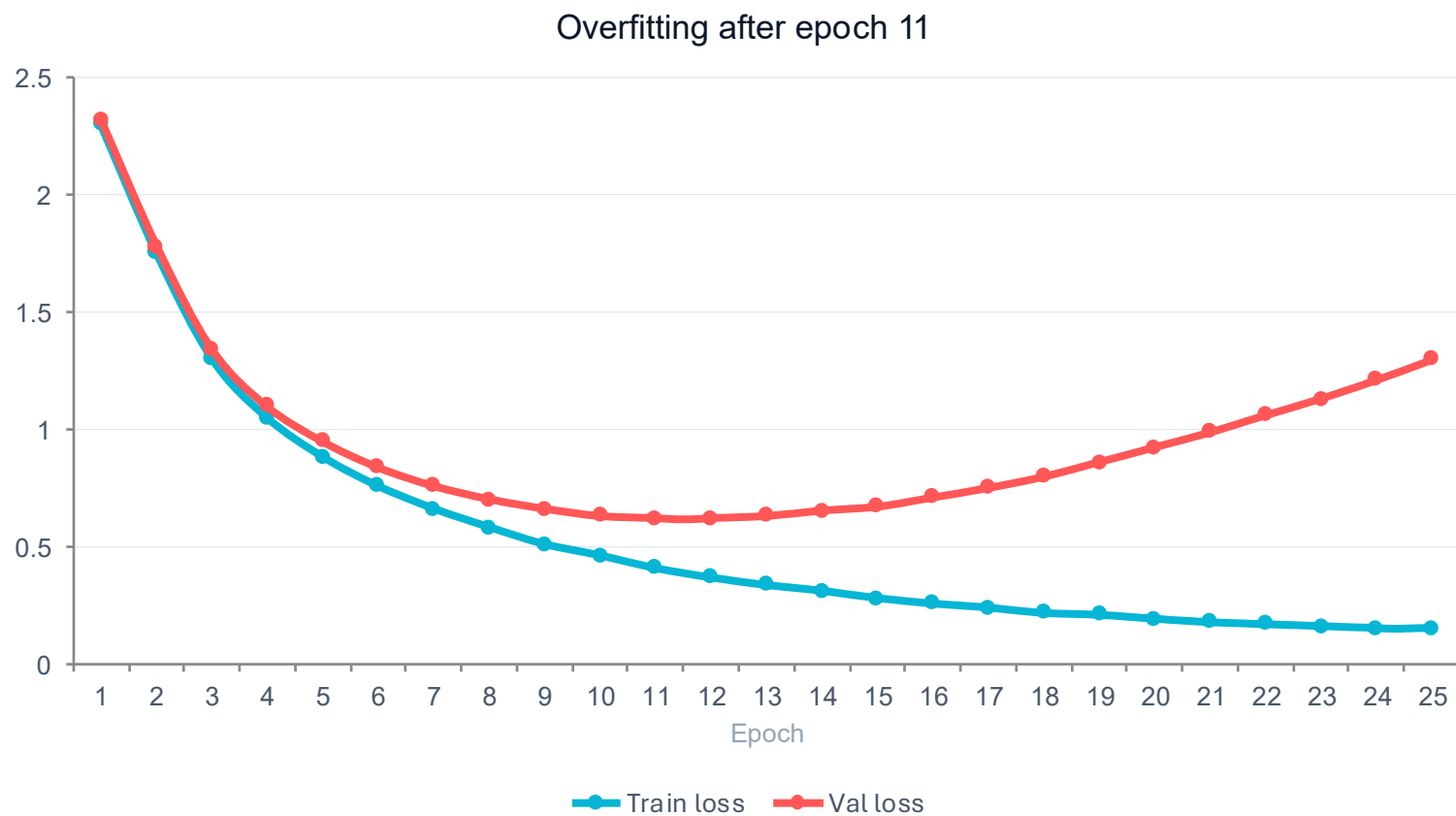


Weight distributions

Change over training. Should evolve, not collapse to zero.

Demo

Diagnosing Loss Curves



Both decrease, gap stable

Healthy. Keep training.

Train ↓, Val ↑ after epoch K

Overfitting. Add regularization or early-stop at K.

Both flat at high loss

Underfitting. Larger model, longer schedule, or higher LR.

Wild oscillations

LR too high, batch too small, or bad init.

Demo

Weights & Biases

Experiment tracking dashboard

WHY W&B?



Logs metrics, gradients, system stats — all in one place



Automatically save model checkpoints alongside the run, code, and config that produced them



Compares dozens of runs side-by-side on a single dashboard

```
setup.sh / train.py
```

```
$ pip install wandb  
$ wandb login
```

```
import wandb
```

```
wandb.init(  
    project='fashion-mnist',  
    name='cnn-baseline',  
    config={  
        'lr': 1e-3, 'batch_size': 64,  
        'epochs': 20, 'arch': 'CNN',  
    },  
)
```

```
wandb.watch(model, log='all', log_freq=100)
```

Log, watch, save

```
train_loop.py

for epoch in range(cfg.epochs):
    model.train()
    for x, y in train_loader:
        x, y = x.to(dev), y.to(dev)
        logits = model(x)
        loss = criterion(logits, y)
        opt.zero_grad(); loss.backward(); opt.step()

        wandb.log({'train/loss': loss.item(),
                  'lr': opt.param_groups[0]['lr']})

    val_loss, val_acc = evaluate(model, val_loader)
    wandb.log({'val/loss': val_loss,
              'val/acc': val_acc, 'epoch': epoch})

    # checkpoint best model as a W&B artifact
    if val_acc > best_acc:
        torch.save(model.state_dict(), 'best.pt')
        art = wandb.Artifact('model', type='model')
        art.add_file('best.pt') # attaches the checkpoint file to the artifact
        wandb.log_artifact(art) # uploads it to W&B and records the link between this run and
                                # this artifact. That's the lineage piece.
```

Training loop

Standard forward, backward, step.

wandb.log

Scalar metrics; one call per step or epoch.

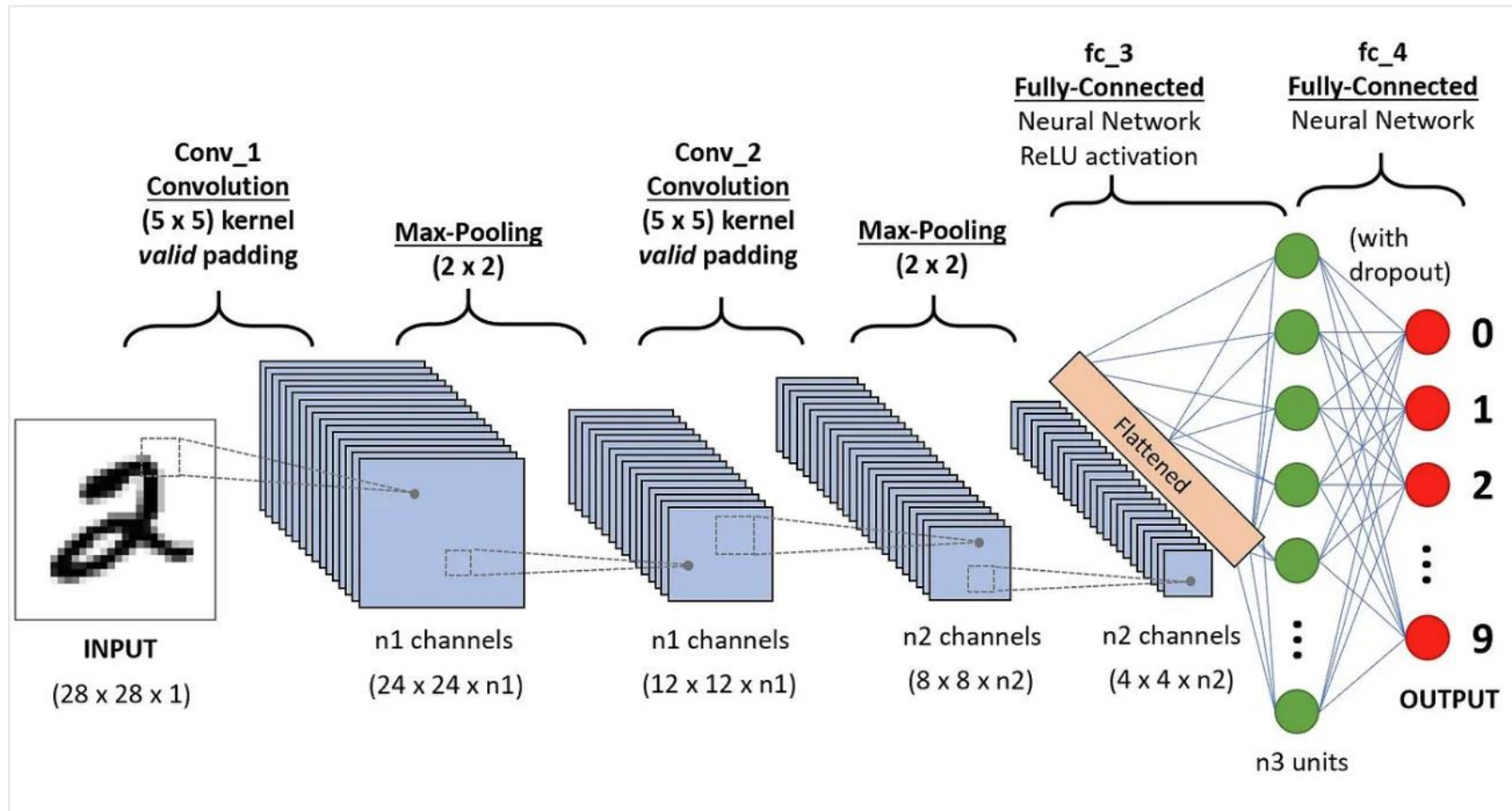
Artifact

Versioned checkpoint tied to this run. In the W&B UI, you can click from the artifact to the run that produced it, see the exact hyperparameters, code, and metrics, and vice versa.

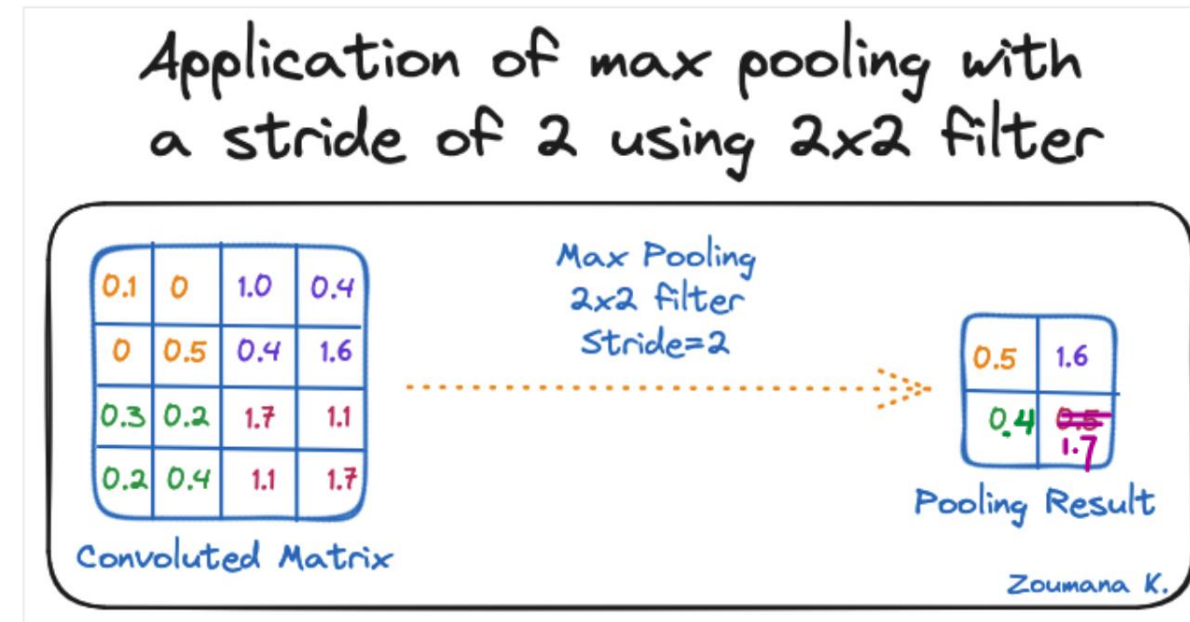
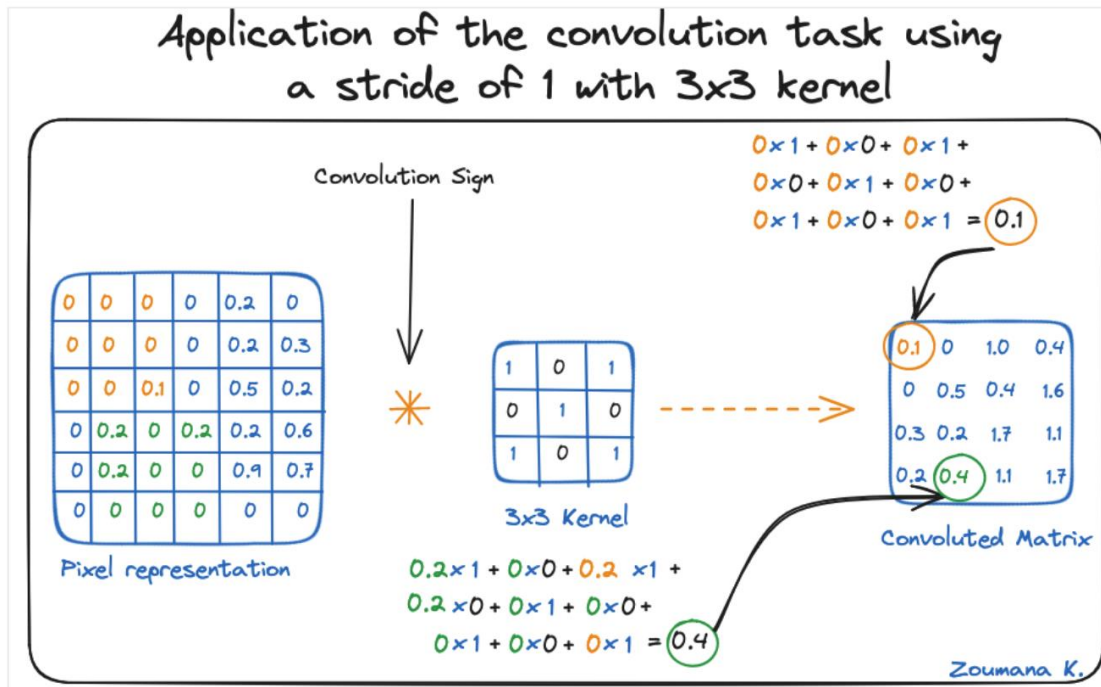
FashionMNIST: MLP vs CNN

Comparing experiment runs using W&B

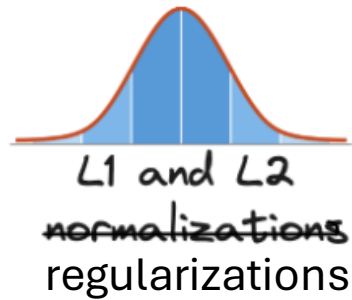
Convolutional Neural Network (CNN)



Convolution and Max pooling



7 Strategies to Mitigate Overfitting in CNNs

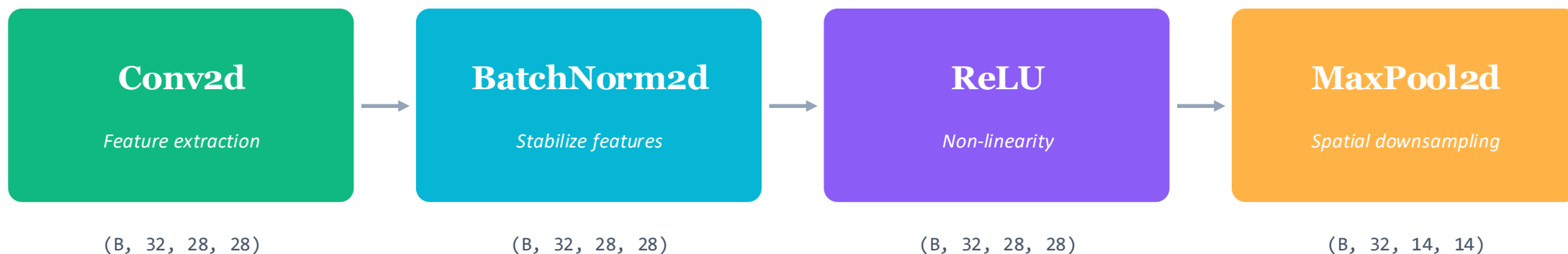


Zoumana K.

<https://www.datacamp.com/tutorial/introduction-to-convolutional-neural-networks-cnns>

The Conv → BN → ReLU → Pool block

Modern CNNs are built from a repeated four-step block. Each step has a specific job; the order matters.



Conv2d(in, out, k)

Slides 'out' filters of size $k \times k \times \text{in}$ across the image. Each filter learns one local pattern (e.g. edge, corner, texture).

BatchNorm2d(out)

Normalizes each feature map across the batch and spatial dims. Can use higher learning rates and reduces sensitivity to weight initialization.

ReLU

$\max(0, x)$. Cheap, non-saturating, applied after BN.

MaxPool2d(2)

Halves spatial resolution by taking the max in each 2×2 window. Local invariance, reduces computation, expands receptive field.

Repeat this block, doubling channels and halving resolution at each stage.

Data augmentation

Augmentation applies label-preserving random transforms each time a sample is drawn. The model never sees the same image twice.

transforms.py

```
from torchvision import transforms
```

```
train_tfm = transforms.Compose([
    # geometry – match training to test variation
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomCrop(28, padding=2),
    transforms.RandomRotation(degrees=10),
```

```
    # photometric – robustness to brightness
    transforms.ColorJitter(brightness=0.2,
                           contrast=0.2),
```

```
    transforms.ToTensor(),
    transforms.Normalize((0.286,), (0.353,)),
])
```

```
# val/test: NO augmentation – only ToTensor + Norm
val_tfm = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.286,), (0.353,)),
])
```

Label-preserving

Data augmentation preserve labels.

On-the-fly

Transforms run inside `__getitem__`, so each epoch sees a fresh draw. Effectively a much larger training set with no extra disk storage.

Train only, never evaluation

Validation must measure performance on real data.

Match the test distribution

Pick transforms that mimic real variation you expect to see.

Summary

- Dataset and Dataloader
- Monitor gradients and activations across layers during training
- Strategies for stabilize gradients and regularization
- Monitor and visualize model performance
- Convolutional neural network
- Data augmentation

Homework (optional)

- Train a small CNN on FashionMNIST under three conditions: (a) no normalization, (b) BatchNorm after each conv/linear layer, (c) LayerNorm after each conv/linear layer. For each condition:
 - Use forward hooks to record the pre-activation values at 2–3 chosen layers every N training steps.
 - Visualize the histograms of these activations and weights using W&B.
 - Plot the mean and standard deviation of the pre-activations over training steps.